What is different in our approach — and the power / area gap

A focused comparison of our SNN packet classifier against existing methodologies: what we do differently, and how that shows up in power and area. Numbers are from the cited papers and from our own tool reports (`docs/reports.md`).

1. The one-line difference

> Existing work runs the classifier **off the packet's path** — in software, on a separate neuromorphic chip fed by software, or (for in-network ML) in a switch with a **non-spiking** model. **We put a spiking neural network *inside the NIC's receive datapath*, so a malicious frame is erased before it is ever written to host memory (DMA).** Because the model is spiking, it uses **no multipliers**, which is what makes it small and low-power.

Three concrete differences follow: **where** it runs, **what** model it is, and **how big** it is.

2. Difference 1 — where the classifier sits (the main novelty)

approach	**where the decision is made**	**malicious packet reaches…?**
Classic IDS (Snort/software)	CPU, after the OS receives the packet	the OS, the sockets, apps
SNN IDS on a neuromorphic chip — Alom & Taha [C1]	a separate TrueNorth chip, **fed by software**	the OS first (software extracts & sends features)
SNN IDS in software/sim — Zhou & Li [C2]	a CPU/GPU or simulator	the OS
In-network ML — Taurus [C3]	inside a **switch** pipeline (non-spiking)	forwarded/dropped in the switch, not at the host NIC
Ours	**inside the NIC RX datapath, before the host FIFO/DMA**	**never — dropped in hardware before the OS sees it**

This "drop-before-DMA" placement is the differentiator. Supporting it required a store-and-forward buffer with commit/rollback, in-order verdicts, and a design that never back-pressures the MAC — all verified (T1–T10, `docs/verification_tables.md`). To our knowledge (papers in `Papers/`), no prior SNN IDS is fused into the NIC datapath at line rate.

3. Difference 2 — a spiking, multiplier-free model

	conventional (CNN/DNN) accelerators	**our SNN**
core operation	multiply-accumulate (MAC)	**add-the-weight-or-not** (spike × weight)
multipliers / DSPs	many (see §5)	**0**
inputs	numbers	1-bit spikes (64 header

	conventional (CNN/DNN) accelerators	our SNN
		features)
work per input	all neurons every pass	only the **active** inputs (event-driven; ~13 of 64)
memory over time	none (needs extra state)	built in (leak + 32-frame rate window) → catches floods/scans

Taurus [C3] is in-network like us, but its models are **non-spiking** (DNN/SVM/k-means) and so are multiply-heavy. Our spiking choice is exactly what removes the multipliers.

4. Difference 3 — scale (task-matched, tiny)

We classify **packet headers** (64 binary features), not images, so the network is tiny: **64 → 16 → 2**, 1.1 KB of int8 weights, 8 time steps. Most published SNN FPGA accelerators target **image** datasets (MNIST/SVHN/CIFAR), which need far larger networks, buffers and many DSPs. Smaller task + multiplier-free model = much smaller hardware.

5. The power & area gap (numbers)

> **Caveat:** these works target *different workloads* (mostly image classification or whole-switch ML), so this is an order-of-magnitude comparison of hardware cost, not a same-benchmark result. It shows where our design sits.

design	platform	area	power	multipliers/DSP
Ours — `snn_classifier`	sky130 ASIC (P&R)	**1.28 mm²**	**125 mW** @100 MHz (est.)	**0**
Ours — full chip	Cyclone V FPGA	**6 % ALMs** (2,378), 8 BRAM	not measured	**0 DSP**
Ours — full chip	Tang Nano 9K	7,035 LUT4 (81 %), 6 BSRAM	—	**0 DSP**
FPGA SNN accel. [C4]	FPGA (image)	—	**0.535 W**	uses DSPs
Low-power SNN FPGA accel.	FPGA (image)	360 DSP (one config)	**1.54 – 3.1 W** (configs)	**360 DSP**
Plagwitz SNN vs CNN [J2]	FPGA (MNIST/SVHN/CIFAR)	69 DSP	watt-scale	69 DSP
Taurus [C3]	switch ASIC	**+3.8 % area** of a ~500–858 mm² switch (≈ 20–30 mm² of ML)	**+2.8 % power** of a ~270 W switch (≈ several W)	many
TrueNorth (used by Alom & Taha	neuromorphic ASIC	1M-neuron chip	**65 mW** (whole chip)	— (crossbar)

design	**platform**	**area**	**power**	**multipliers/DSP**
[C1])				

What the gap shows: - vs **FPGA SNN/CNN accelerators** (watts, tens–hundreds of DSPs): ours is **0 DSP** and milliwatt-class, because the task is header-sized and the model is multiplier-free. This is the direction Plagwitz et al. [J2] identify — SNNs beat CNNs on hardware cost **when they are sparse and multiplier-free**, which our design is by construction. - vs **in-switch ML** (Taurus [C3], ~20–30 mm² of ML logic inside a 270 W-class switch): ours is **1.28 mm²** and milliwatts, and sits at the **NIC** (per-host) rather than the switch, and is **spiking** rather than DNN. - vs **neuromorphic ASIC** (TrueNorth, 65 mW for a *million* neurons): similar milliwatt class, but TrueNorth is a general 1M-neuron fabric sitting **off** the data path; ours is a fixed 18-neuron datapath **in** the NIC, so it is far smaller and needs no software to feed it.

Where the power/area goes in *our* design (`docs/reports.md`): ~75 % of the ASIC area is the two flop-based memories (2 KiB packet buffer + weights), and most of the estimated power is the clock toggling those flip-flops — both shrink substantially with SRAM macros and clock-gating (the planned next step), which would widen the gap further in our favour.

6. Summary of differences

dimension	**existing methods**	**ours**
placement	off-path / software / switch	**in the NIC RX datapath (drop-before-DMA)**
model	CNN/DNN (MACs) or off-path SNN	**in-line SNN, event-driven**
multipliers	many DSPs	**0**
size	watts, tens–hundreds of DSPs, or mm²-in-a-switch	**1.28 mm² / 125 mW / 0 DSP / 6 % of a small FPGA**
one engine, 3 deployments, bit-exact	—	**ASIC + Tang Nano 9K + Chameleon96**

Honest caveats: (1) the comparisons span different workloads, so treat them as order-of-magnitude. (2) Our accuracy is still on synthetic traffic (`docs/training.md`); published SNN-IDS on *real* data is ≈ 90 % [C1]. (3) Our power is a sky130 estimate; FPGA and Cadence 90 nm power are not yet measured.

References

Journal - **[J1]** Neftci, Mostafa, Zenke, “Surrogate Gradient Learning in Spiking Neural Networks,” *IEEE Signal Processing Magazine*, 2019. - **[J2]** Plagwitz, Hannig, Teich, Keszöcze, “To Spike or Not to Spike? A Quantitative Comparison of SNN and CNN FPGA Implementations,” *ACM Trans. Embedded Computing Systems*, 2023. - **[J3]** Na, Kim, “An Energy-Efficient Neuromorphic Processor Using Unified Refractory Control-Based NoC for Edge AI,” *Electronics (MDPI)*, 2025. - **[J4]** “Algorithm–hardware co-design of neuromorphic networks with dual memory pathways,” *Nature Machine Intelligence*, 2026.

Conference - **[C1]** Alom, Taha, “Network Intrusion Detection for Cyber Security on

Neuromorphic Computing System," *IEEE IJCNN*, 2017. - **[C2]** Zhou, Li, "Spiking Neural Networks with Single-Spike Temporal-Coded Neurons for Network Intrusion Detection," *IEEE ICPR*, 2020. - **[C3]** Swamy et al., "Taurus: A Data Plane Architecture for Per-Packet ML," *ACM ASPLOS*, 2022. - **[C4]** "Efficient Hardware Acceleration of Spiking Neural Networks using FPGA," *IEEE VTC2024-Spring*, 2024.